

Assignment 2: TinyGPT - Implementation and Analysis Report

Naveen Manikandan | UBIT: 50625386

R.1 Training Curve

TinyGPT was trained for 5 epochs on the TinyShakespeare character-level corpus using the baseline configuration: block size 128, embedding dimension 64, 4 attention heads, 4 transformer layers, MLP hidden dimension 128, dropout 0.1, learning rate $3e-4$, batch size 64, and 200 gradient steps per epoch. The model has 145,344 trainable parameters. Both training and validation loss are plotted below against epoch from training_log.json.

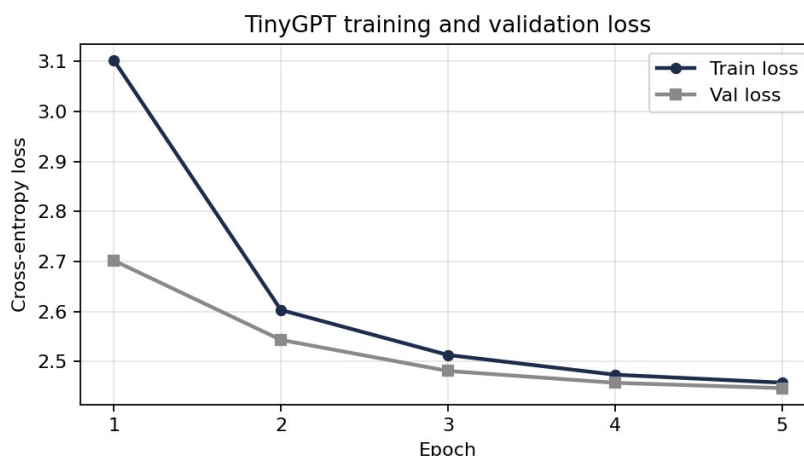


Figure 1: Training and validation cross-entropy loss across the 5 training epochs.

The model is learning normally and is not overfitting. Training loss dropped from 3.102 at epoch 1 to 2.4579 at epoch 5, and validation loss dropped from 2.7019 at epoch 1 to 2.447 at epoch 5. Both curves fall together and flatten out at almost the same level, which is the signature of healthy learning rather than overfitting.

The biggest epoch-to-epoch improvement happens from epoch 1 to epoch 2. Validation loss falls by roughly 0.16 between epoch 1 and epoch 2 (2.7019 to 2.5434), then the gains shrink steadily: about 0.06 from epoch 2 to 3, 0.024 from epoch 3 to 4 and only about 0.011 from epoch 4 to 5. The curve is clearly converging. One detail worth noting is that at epoch 1 the training loss (3.102) is higher than the validation loss (2.7019). This is expected here and is not a

contradiction. Training loss is averaged over the whole epoch including the very early high-loss steps when the model was still close to random, while validation loss is measured only at the end of the epoch once the weights have already improved. From epoch 2 onward the ordering becomes normal, with training loss sitting just below validation loss and the gap staying very small (around 0.01 to 0.06). That small, stable gap is the main evidence against overfitting. An overfitting model would show validation loss rising while training loss kept falling and that never happens here.

If anything, the model is slightly underfitting in the sense that 5 epochs and 200 steps per epoch is a short schedule. The validation curve is still trending downward at epoch 5, just very slowly, so more training would likely yield a small additional improvement. But within this budget the model is behaving exactly as it should.

R.2 Generated Text Analysis

The samples below are generated at temperature 1.0 from the epoch 1, epoch 3, epoch 5 checkpoints, all conditioned on the prompt "HAMLET:".

Epoch 1 (T = 1.0):

```
HAMLET:\nllly ayeJl mingo y ot f opkevars osireaghaldoweselr X\nNlt monm uhe tha yN\nAdulathar pou, ax  
reno?nKuis y ousofaryre soms ! RThuwhfhen,\n\n$UOCKI.ho?t,\n$CB\nMVTONLboGd nand ted WCref  
ysipouHONZX! ed areln cdd l-ANHSU:erenioeg ant, jE;\nOSDAYone mh ITESSAOuscld,\nEusce h\nEX\nNAMA  
F  
st,y d l fotime bentor, 3PK",
```

At epoch 1 the output is essentially noise. There are almost no real English words, just random character clusters like "opkevars" and "osireaghaldoweselr" and the model freely emits stray capitals and symbols mid-word ("Nlt", "yN", "RThuwhfhen", "X"). It has picked up only the most basic statistics: that spaces and newlines exist, and that text is broken into short chunks. It has no sense of which letters go together.

Epoch 3 (T = 1.0):

```
HAMLET:\nO yeand, my tanou is athhy tou, kerimiow f A yignlCy ot,\nTheeour.\nANISWATH fo mar war tncoigr be  
kesMG he WENJuar,\nWard u dmenatullis nspas frun pam aKuled G foorins id,\nceonaint, iro otond &love  
w.\nWighe swelde han?\nDisthe t alrn &osthachevede-m, w blitQunghitZar nde f geve rods  
gin?\n\n\n\nweddoucit ba",
```

By epoch 3 the model begins to produce recognisable English words. Real words begin to appear in scattered places, such as "my", "is", "war", "be", and "he" and the near word "tou" (clearly aiming at "to" or "you"). The fragment "my tanou is" has correct English word shape and rhythm even though "tanou" is not a real word. So, epoch 3 is the point where individual words become recognizable, even though the surrounding text is still mostly gibberish, and the lines do not form sentences. The model has also clearly learned that capitalized words on their own line are a thing, since it produces "ANISWATH" and "WENJuar" in roughly the position a speaker name would appear.

Epoch 5 (T = 1.0):

```
HAMLET:\nHich thar, himo hafrt lin an-t furepourg y seothmyor.\nMat hens Senndas ngawais s ptede neas  
ow;\nDUAngheathithe! nd th oourdy CYonoe ouarsn-nor arareit f bl, n\nYo u t no enoweavisy fa?\nHur'therther  
oangenost wat, d hangld l nd woraiish men.\n\nOMENNEORO!\nAROd d aloul ichoned me,\nBRS:\nNothtecat  
thit ang",
```

By epoch 5 the quality has improved again but the model still does not produce full grammatical sentences. What it does produce is text that looks like English at a glance: correct word lengths, plausible letter combinations and real words appearing more often ("me", "s", "thar" close to "that", "hens" close to "hence"). More importantly the structure of a Shakespeare script is now visible. The line "BRS:" is not a real speaker name, but it appears in the same format as a Shakespeare-style speaker label. Similarly, "OMENNEORO!" is an all-caps exclamation of the kind the training text is full of. Punctuation is also used in sensible places, with commas inside lines and semicolons and periods ending them.

So, the progression is clear. Epoch 1 is random characters, epoch 3 is where recognizable English words first appear and epoch 5 produces word-shaped text with correct script formatting but no full sentences. At no checkpoint does the model generate a complete, grammatical sentence, which is the expected ceiling for a 145K-parameter character-level model trained for only 5 epochs.

R.3 Temperature Analysis

The three samples below all come from the same epoch-5 checkpoint and the same "HAMLET:" prompt. Only the sampling temperature changes.

T = 0.5	HAMLET:\nThe sere se ar the chice wind th thendit thar the theror'd owe the reed thand thethint hel me se heando thely the athere he tous he him,\nCourere at hindes wis t band fak withend ghe ond, nour y anoras touris s me he there the the onouthe anourd the\nl wnd he;\nl cenou he s amae me he mathe t sethin wo",
T = 1.0	"HAMLET:\nHich thar, himo hafrt lin an-t furepourg y seothmyor.\nMat hens Senndas ngawais s ptede neas ow;\nDUAngheathithe! nd th oourdy CYonoe ouarsn-nor arareit f bl, n\nYo u t no enoweavisy fa?\nHur'therther oangenost wat, d hangld l nd woraiish men.\n\nOMENNEORO!\nAROd d aloul ichoned me,\nBRS:\nNohtecat thit ang",
T = 1.5	"HAMLET:\nKiu,Noond wown YYil koor.\nf aEKY:'lf arylsschGt l,\nbig:\nMirtceO.Gl'- Prn!\nPos meV?\nEUZY.\nRSRANfbmvenoy, ptt d3REFWF stomo.\n\nFd, nvi\$epis:\n\nG\nSemeOq, WEUJ&O:\nvulf, gortech whechey, g,, Hu&eapri&le\nCl.\nHir.\nH--aoPzTo!!\nANES:UNICx:QO e.\nOChecdg ttffudrauquth IENans.LZVot 'tay IFK\nJ\$Ruk, gat'T, tiqULs-A"} }

Table 1: Epoch-5 output at three sampling temperatures.

As temperature rises the output gets more diverse but less coherent. At $T = 0.5$ the text is the most repetitive and the most word-like: "the" appears over and over ("ar the chice wind th thendit thar the theror'd"), the spacing is regular, and there are almost no stray capitals or symbols. At $T = 1.0$ the text is more varied, with longer and less predictable words and more punctuation, but it also starts dropping in quality with odd fragments like "ngawais" and "seothmyor". At $T = 1.5$ the output collapses into near-noise: random capital runs ("YYil", "EKY", "RSRANfbmvenoy"), digits and stray punctuation such as apostrophes, colons, commas, and fragments like "d3REFWF" and barely any real words at all.

This follows directly from how temperature reshapes the probability distribution before sampling, as described in Section 3.7. The model produces a logit vector. During sampling, the logits are divided by the temperature before applying softmax, which turns them into probabilities. Dividing by a temperature below 1 sharpens the distribution: the highest-probability character gets even more probability mass, and the unlikely characters get squeezed toward zero. So, at $T = 0.5$ the model almost always picks its single most confident next character. Since the most confident character after "th" is usually "e", the model keeps generating "the" and gets stuck in safe, repetitive loops.

Dividing by a temperature above 1 does the opposite. It shrinks the logit gaps and flattens the distribution toward uniform, so low-probability characters, including capitals, digits and symbols that rarely follow the current context, get a real chance of being sampled. At $T = 1.5$ the distribution is flat enough that the model frequently

samples characters its own statistics say are unlikely, which is why coherence breaks down. $T = 1.0$ leaves the distribution unchanged and sits in between: more variety than $T = 0.5$ because the model is not forced onto its top pick every step, but more structure than $T = 1.5$ because it is still mostly following its learned probabilities. This is the core trade-off. Low temperature buys coherence at the cost of repetition, high temperature buys diversity at the cost of coherence.

R.4 Reflection

Even with only 145,344 parameters and 5 epochs, TinyGPT has clearly learned several real properties of English text. The most basic property is character n-gram statistics. By epoch 5 the generated text is built from plausible letter sequences like "thar", "hens" and "thand" rather than random clusters, which means the model has learned which characters tend to follow which. It has also learned word boundaries: spaces are placed at sensible intervals and the generated tokens have realistic word lengths instead of running together or fragmenting. It has picked up some capitalization rules too, since the worst capital-in-the-middle-of-a-word behavior from epoch 1 mostly disappears by epoch 5 and capitals tend to land at the start of lines.

The most interesting thing it learned is higher-level structure specific to the Shakespeare corpus. The model reproduces the dialogue formatting of a play: short all-caps speaker labels on their own line followed by a colon, like "BRS:" in the epoch-5 sample and all-caps exclamations like "OMENNEORO!". It learned this purely from character statistics, because in the training text a newline is very often followed by a run of capitals then a colon and another newline. Punctuation is also used in roughly the right positions, with commas inside lines and periods or semicolons ending them.

The limitations are equally clear. The model never produces a full grammatical sentence and never produces a long stretch of correct real words. It has no real vocabulary, only character-level habits, so it generates word-shaped strings like "tanou" and "furepourg" that look right but mean nothing. It has no grammar, no syntax and no memory of meaning across a line. The 128-character context window and the tiny 64-dimensional embedding simply do not have the capacity to represent words as units or to track sentence structure.

A larger model would overcome these in direct ways. More layers and a wider embedding would give the model the capacity to represent longer-range dependencies and to model grammar rather than just local letter transitions, which is the difference between word-shaped noise and real sentences. The single biggest change, though, would be subword tokenization. A character-level model must spend all its capacity just learning how to spell, since it predicts one letter at a time. A subword tokenizer gives the model whole common words and word-pieces as single tokens, so spelling is essentially free, and the model can spend its capacity on word order and meaning instead. Combined with a longer context window and more training, that is what moves a model from "recognizable English words" to actual coherent English.